

多智能体隐式指令注入防御系统

—— 项目启动内部说明文档（补充：强调可落地性）

一、项目背景

随着多智能体系统（Multi-Agent System, MAS）的广泛应用，多个 Agent 之间的协作逐渐成为主流架构模式。然而当前主流框架默认：

- Agent 之间通信可信
- 内部消息无需权限验证
- 不对跨角色调用进行安全审计

这导致一种新型安全风险：

攻击者通过低权限 Agent 间接操控高权限 Agent，实现隐式越权访问。

该问题属于：

- 隐式指令注入（Implicit Prompt Injection）
- 跨 Agent 权限提升（Cross-Agent Privilege Escalation）

本项目旨在构建一个：

面向多智能体协作链的零信任安全防御框架
且具备真实可部署能力，而非仅理论验证。

二、项目核心目标

我们要做到的不仅是“能演示”，而是：

- 架构可部署
- 规则可维护

- 3. 策略可扩展
- 4. 性能可接受
- 5. 安全逻辑可解释

最终目标：

- 无防御时攻击成功
- 开启防御后攻击失败
- 系统输出可解释的风险分析
- 日志可用于事后审计

三、可落地性原则（非常关键）

本项目必须遵循以下落地原则：

1 不依赖大规模模型训练

我们不会：

- 重新训练大模型
- 进行复杂对抗训练

原因：

- 不可控
- 成本高
- 评估困难

我们采用：

规则 + 轻量语义识别 + 权限模型 的组合架构

2 不侵入底层 Agent 框架

系统设计为：

通信中间层（Middleware）

特点：

- 不修改 Agent 内部逻辑
- 不依赖特定框架
- 可插拔部署

这意味着未来可以：

- 接入 AutoGen
- 接入 CrewAI
- 接入企业内部 MAS 系统

3 权限模型必须工程可维护

策略文件采用：

- YAML 或 JSON 格式

要求：

- 可人工阅读
- 可版本管理
- 可签名验证
- 可动态更新

不能写成：

- 难以理解的硬编码逻辑

4 性能必须可接受

审核 Agent 不能成为瓶颈。

设计原则：

- 单次审核延迟可控
- 支持异步处理
- 支持缓存策略

目标：

系统可在真实业务流中运行，而非仅实验室演示。

5 审计日志必须真实可用

日志要求：

- 记录完整调用链
- 记录权限决策依据
- 支持哈希链防篡改

这使系统具备：

- 事后追责能力
- 合规审计能力

四、系统架构设计（可部署版本）

User

↓

Agent A

↓

Security Middleware（审核层）

↓

Agent B

审核层职责：

1. 拦截跨 Agent 请求
2. 解析请求内容

3. 判断意图与数据类型
4. 检查角色权限
5. 记录日志
6. 决策是否放行

该层可作为：

- 独立服务（FastAPI）
- 网关插件
- 企业内部微服务

五、核心模块划分（工程实现角度）

1 权限管理模块

- 角色定义
- 能力定义
- 敏感资源分级
- 权限匹配算法

2 意图识别模块

目标：

- 判断是否请求敏感数据
- 判断是否执行敏感操作

实现方式：

- 规则匹配
- LLM 分类辅助
- 关键词 + 上下文分析

3 协作路径分析模块

功能：

- 记录调用链
- 检查是否存在跨角色越权
- 分析多跳风险

4 审计与日志模块

功能：

- JSON 日志记录
- 哈希链防篡改
- 风险评分输出

六、关键落地挑战

⚠ 审核 Agent 自身可信问题

必须：

- 策略文件签名验证
- 防绕过设计
- 部署独立服务

否则系统失去安全意义。

⚠ 误判率控制

若误判过高：

- 系统不可用

必须做到：

- 可解释决策
- 支持人工白名单
- 支持策略调整

⚠ 可扩展性

设计必须支持：

- 新增 Agent
- 新增角色
- 新增资源类型

而不需要重构系统。

七、创新点（在可落地前提下）

1. 面向多智能体通信层的零信任架构
2. 协作路径级越权检测
3. 审核 Agent 可信机制设计
4. 可部署的中间层安全方案

我们强调：

创新 ≠ 复杂

创新 = 结构性安全改进 + 工程可落地

八、我们最终要呈现的价值

不是一个概念模型。

而是：

- 可运行系统
- 可攻击
- 可防御
- 可解释
- 可扩展
- 可部署

如果未来企业部署多 Agent 系统：

我们的架构可以直接作为安全网关接入。

九、总结

本项目定位：

多智能体协作场景下的工程级安全防御系统

核心原则：

- 零信任
- 最小权限
- 可审计
- 可部署
- 可扩展

我们要做的不是“研究模型防攻击”，
而是“设计一个真实可用的多智能体安全架构”。